

Result	Time	Cycles	Regs	GPU	SM Frequency	CC	Process
Current	521	12.53 msecond	12,682,227	31	0 - NVIDIA GeForce RTX 2060	1.01 cycle/nsecond	7.5 [25720] project_canny

GPU Speed of Light Throughput

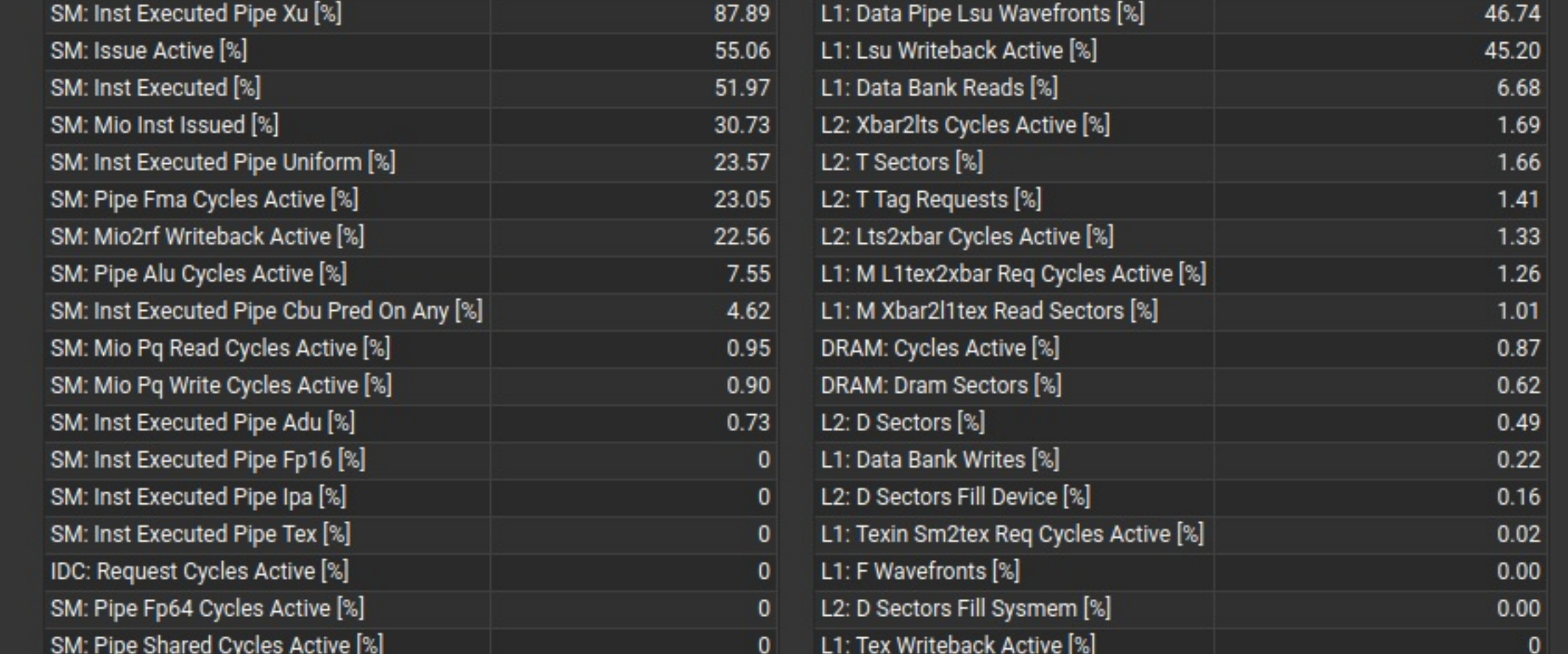
All

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roffline chart.

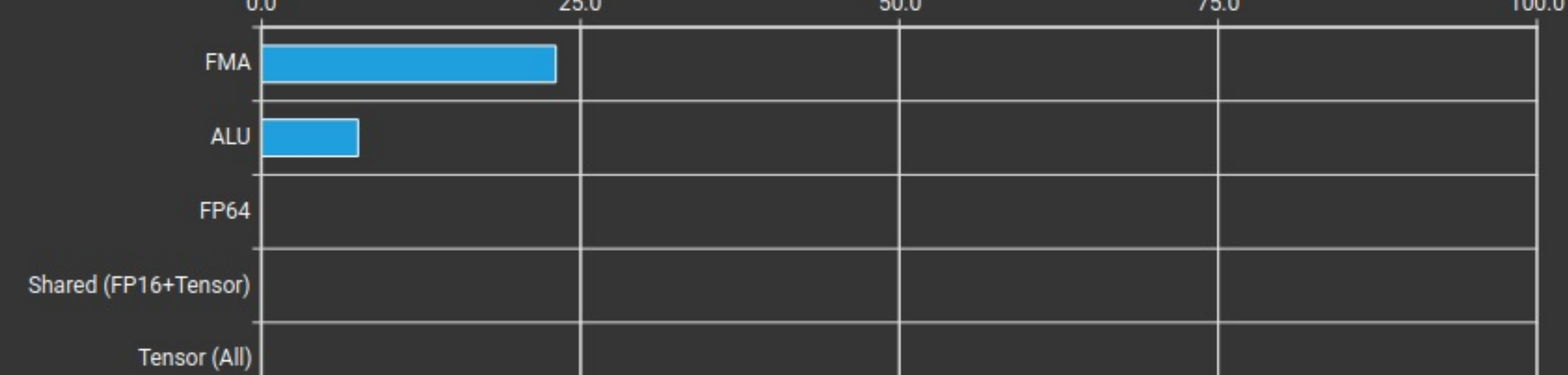
Compute (SM) Throughput [%]	91.47	Duration [msecond]	12.53
Memory Throughput [%]	91.47	Elapsed Cycles [cycle]	12,682,227
L1/TEX Cache Throughput [%]	93.48	SM Active Cycles [cycle]	12,643,171
L2 Cache Throughput [%]	1.69	SM Frequency [cycle/nsecond]	1.01
DRAM Throughput [%]	0.87	DRAM Frequency [cycle/nsecond]	5.49

The kernel is utilizing greater than 80.0% of the available compute or memory performance of the device. To further improve performance, work will likely need to be shifted from the most utilized to another unit. Start by analyzing workloads in the [Compute Workload Analysis](#) section.

The ratio of peak float (fp32) to double (fp64) performance on this device is 32:1. The kernel achieved 22% of this device's fp32 peak performance and 0% of its fp64 peak performance. See the [Kernel Profiling Guide](#) for more details on roffline analysis.



Compute Throughput Breakdown			Memory Throughput Breakdown		
SM: Inst Executed Pipe Lsu [%]	91.47		L1: Lsuin Requests [%]		91.47
SM: Inst Executed Pipe Xu [%]	87.89		L1: Data Pipe Lsu Wavefronts [%]		46.74
SM: Issue Active [%]	55.06		L1: Lsu Writeback Active [%]		45.20
SM: Inst Executed [%]	51.97		L1: Data Bank Reads [%]		6.68
SM: Mio Inst Issued [%]	30.73		L2: Xbar2lts Cycles Active [%]		1.69
SM: Inst Executed Pipe Uniform [%]	23.57		L2: T Sectors [%]		1.66
SM: Pipe Fms Cycles Active [%]	23.05		L2: T Tag Requests [%]		1.41
SM: Mio2f Writeback Active [%]	22.56		L2: Lts2xbar Cycles Active [%]		1.33
SM: Pipe Alu Cycles Active [%]	7.55		L1: M L1tex2xbar Req Cycles Active [%]		1.26
SM: Inst Executed Pipe Cbu Pred On Any [%]	4.62		L1: M Xbar2l1tex Read Sectors [%]		1.01
SM: Mio Pq Read Cycles Active [%]	0.95		DRAM: Cycles Active [%]		0.87
SM: Mio Pq Write Cycles Active [%]	0.90		DRAM: Dram Sectors [%]		0.62
SM: Inst Executed Pipe Adu [%]	0.73		L2: D Sectors [%]		0.49
SM: Inst Executed Pipe Fp16 [%]	0		L1: Data Bank Writes [%]		0.22
SM: Inst Executed Pipe Ipa [%]	0		L2: D Sectors Fill Device [%]		0.16
SM: Inst Executed Pipe Tex [%]	0		L1: Texin Sm2l1tex Req Cycles Active [%]		0.02
IDC: Request Cycles Active [%]	0		L1: F Wavefronts [%]		0.00
SM: Pipe Fp64 Cycles Active [%]	0		L2: D Sectors Fill System [%]		0.00
SM: Pipe Shared Cycles Active [%]	0		L1: Tex Writeback Active [%]		0
SM: Pipe Tensor Cycles Active [%]	0		L2: D Atomic Input Cycles Active [%]		0
			L1: Data Pipe Tex Wavefronts [%]		0



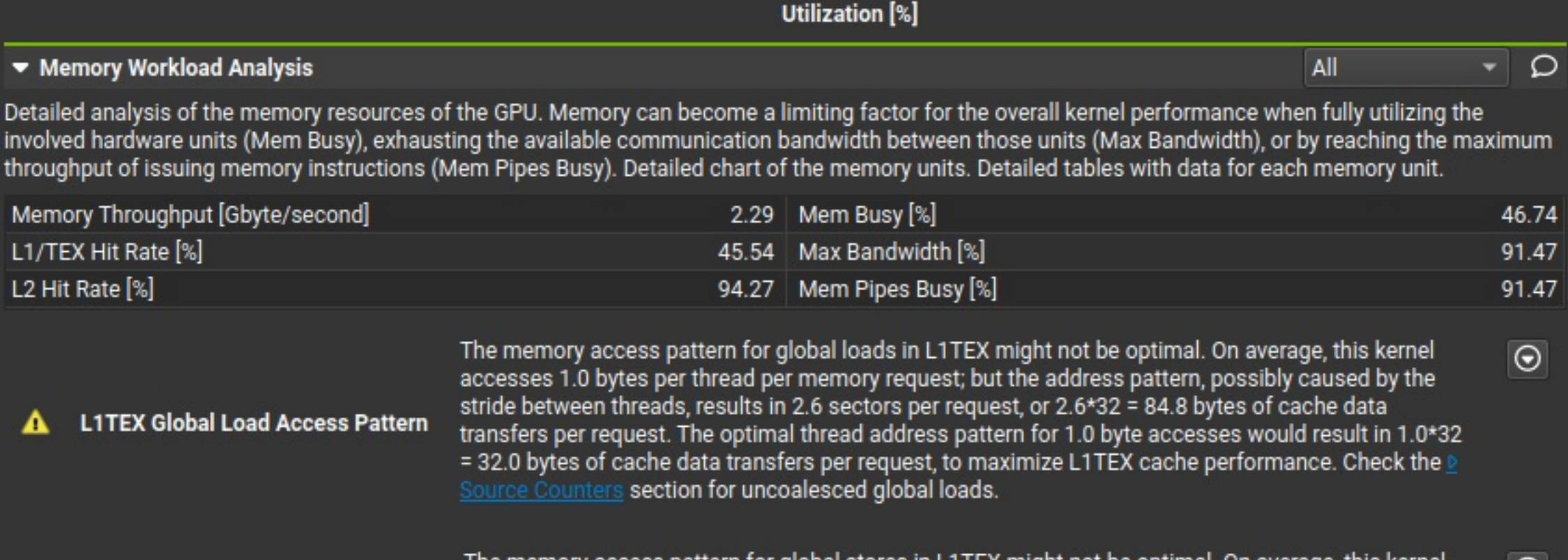
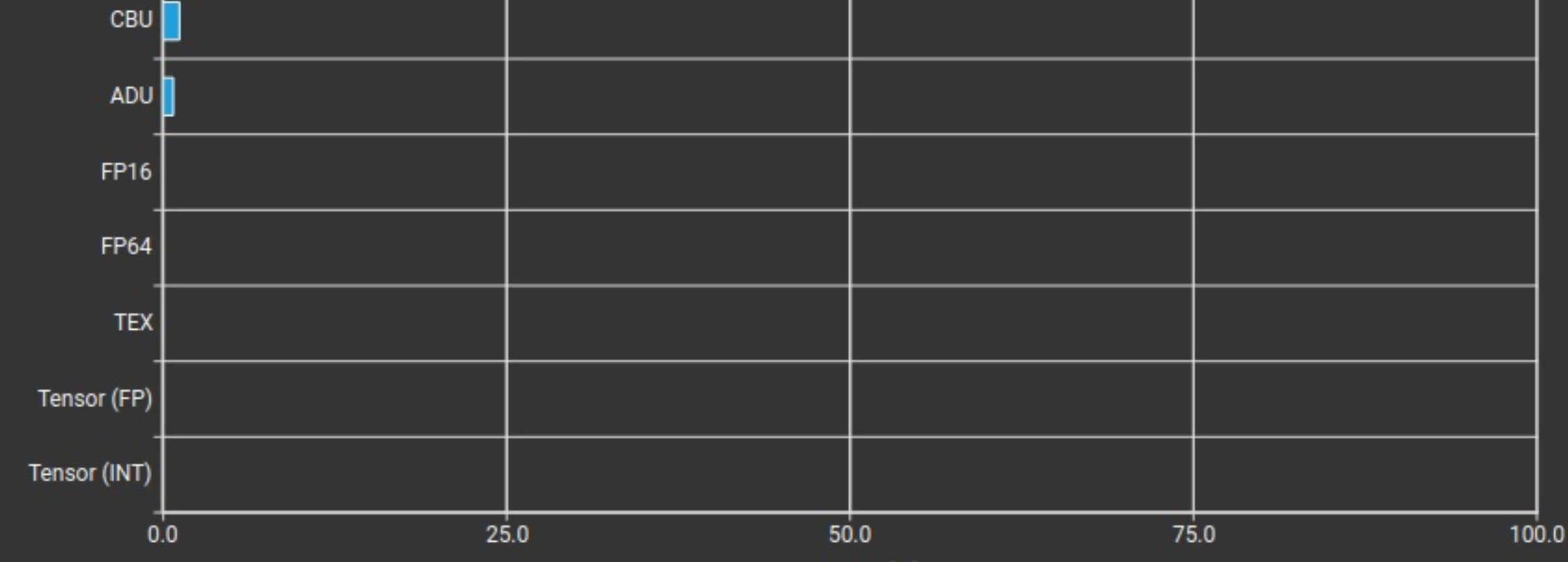
Compute Workload Analysis

All

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed Ipc Elapsed [Inst/cycle]	2.08	SM Busy [%]	87.96
Executed Ipc Active [Inst/cycle]	2.08	Issue Slots Busy [%]	55.10
Issued Ipc Active [Inst/cycle]	2.20		

FMA is the highest-utilized pipeline (23.1%) based on active cycles, taking into account the rates of its different instructions. It accesses 32-bit floating point (FADD, FMUL, FMAD, ...) and integer (IMUL, IMAD) operations. It is well-utilized, but should not be a bottleneck.



Memory Workload Analysis

All

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

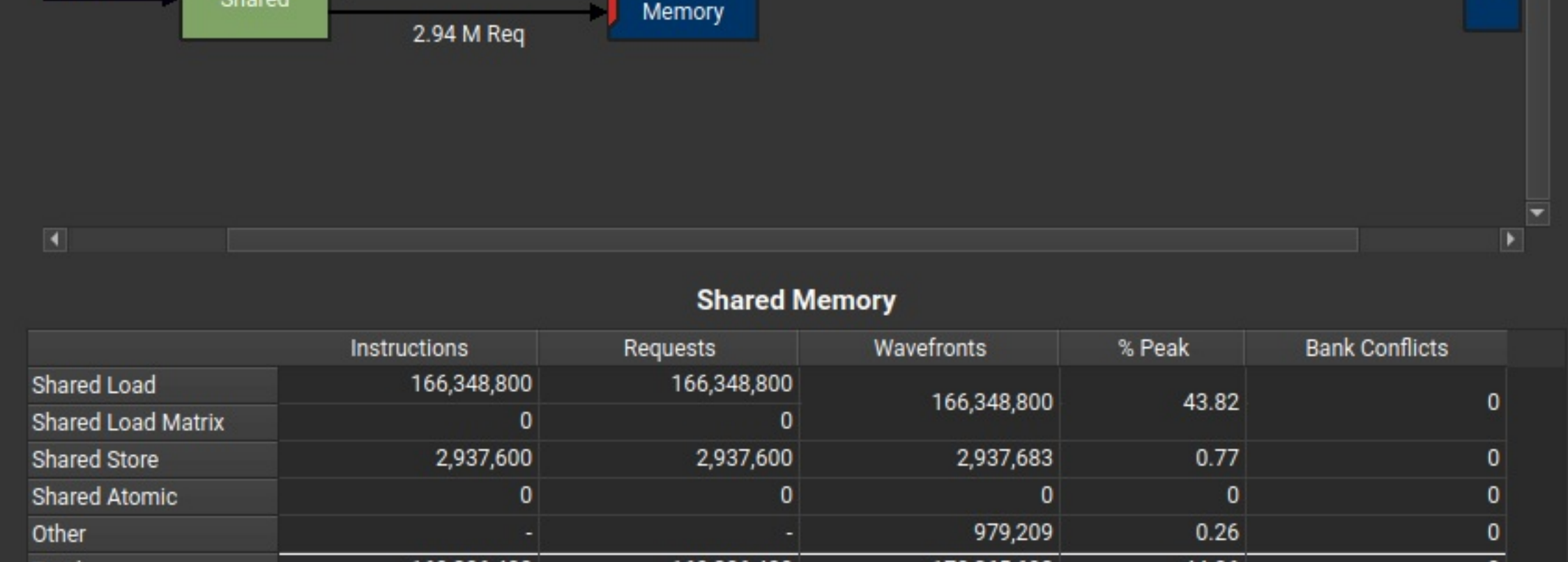
Memory Throughput [Gbyte/second]	2.29	Mem Busy [%]	46.74
L1/TEX Hit Rate [%]	45.54	Max Bandwidth [%]	91.47
L2 Hit Rate [%]	94.27	Mem Pipes Busy [%]	91.47

The memory access pattern for global loads in L1TEX might not be optimal. On average, this kernel accesses 1.0 bytes per thread per memory request, but the address pattern, possibly caused by the stride between threads, results in 2.6 sectors per request, or 2.6*32 = 84.8 bytes of cache data transfers per request. The optimal thread address pattern for 1.0 byte accesses would result in 1.0*32 = 32.0 bytes of cache data transfers per request, to maximize L1TEX cache performance. Check the [Source Counters](#) section for uncoalesced global loads.

The memory access pattern for global stores in L1TEX might not be optimal. On average, this kernel accesses 1.0 bytes per thread per memory request, but the address pattern, possibly caused by the stride between threads, results in 2.0 sectors per request, or 2.0*32 = 64.0 bytes of cache data transfers per request. The optimal thread address pattern for 1.0 byte accesses would result in 1.0*32 = 32.0 bytes of cache data transfers per request, to maximize L1TEX cache performance. Check the [Source Counters](#) section for uncoalesced global stores.

The memory access pattern for loads from L1TEX to L2 is not optimal. The granularity of an L1TEX request to L2 is a 128 byte cache line. That is 4 consecutive 32-byte sectors per L2 request. However, this kernel only accesses an average of 1.0 sectors out of the possible 4 sectors per cache line. Check the [Source Counters](#) section for uncoalesced loads and try to minimize how many cache lines need to be accessed per memory request.

The memory access pattern for stores from L1TEX to L2 is not optimal. The granularity of an L1TEX request to L2 is a 128 byte cache line. That is 4 consecutive 32-byte sectors per L2 request. However, this kernel only accesses an average of 1.0 sectors out of the possible 4 sectors per cache line. Check the [Source Counters](#) section for uncoalesced stores and try to minimize how many cache lines need to be accessed per memory request.



Shared Memory					
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	166,348,800	166,348,800			
Shared Load Matrix	0	0	166,348,800	43.82	0
Shared Store	2,937,600	2,937,600	2,937,683	0.77	0
Shared Atomic	0	0	0	0	0
Other	-	-	979,209	0.26	0
Total	169,286,400	169,286,400	170,265,692	44.86	0

L1/TEX Cache					
	Instructions	Requests	Wavefronts	% Peak	Sectors
Local Load	0	0	0	0	0
Global Load	2,926,868	2,926,868	4,074,006	1.07	7,753,457
Surface Load	0	0	0	0	0
Texture Load	0	0	0	0	0
Global Store	460,800	460,800	657,408	0.17	921,600
Local Store	0	0	0	0	0
Surface Store	0	0	0	0	0
Global Reduction	0	0	0	0	0
Surface Reduction	0	0	0	0	0
Global Atomic ALU	0	0	0	0	0
Global Atomic CAS	0	0	0	0	0
Surface Atomic ALU	0	0	0	0	0
Surface Atomic CAS	0	0	0	0	0
Loads	2,926,868	2,926,868	4,074,006	1.07	7,753,457
Stores	460,800	460,800	657,408	0.17	921,600
Atoms & Reductions	0	0	0	0	0

L2 Cache					
	Requests	Sectors	Sectors/Req	% Peak	Hit Rate
L1/TEX Load	3,127,512	3,844,153	1.23	1.33	88.59
L1/TEX Store	921,600	921,600	1	0.32	100
L1/TEX Atomic ALU	0	0	0	0	0
L1/TEX Atomic CAS	0	0	0	0	0
L1/TEX Reduction	0	0	0	0	0
L1/TEX Total	4,049,246	4,766,739	1.18	1.65	90.70
ECC Total	-	-	-	0	-

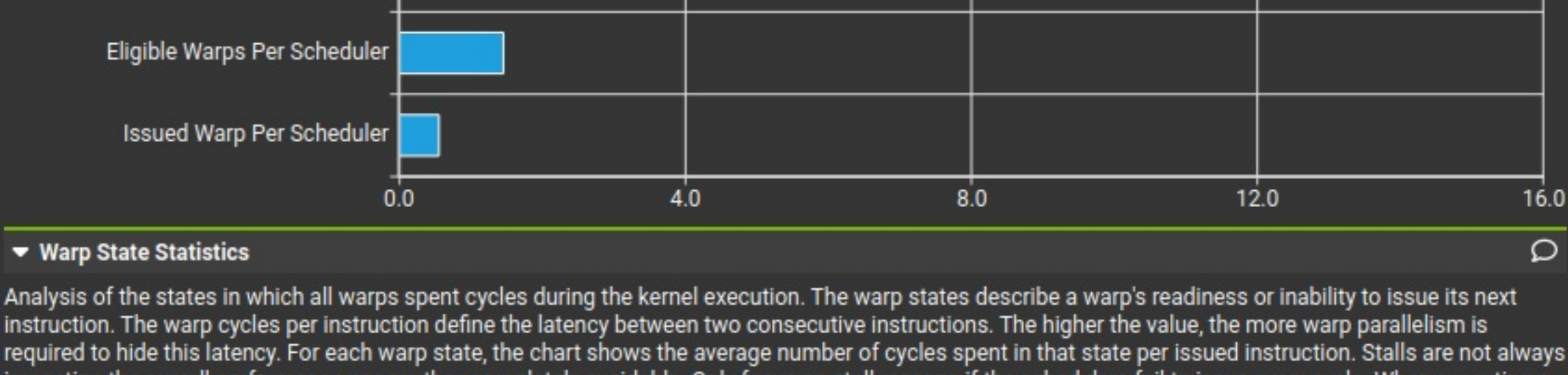
Device Memory			
	Sectors	% Peak	Bytes
Load	475,953	0.46	15,230,496
Store	421,782	0.41	13,497,024
Total	897,735	0.87	28,727,520

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	7.58	No Eligible [%]	44.89
Eligible Warps Per Scheduler [warp]	1.45	One or More Eligible [%]	55.11
Issued Warp Per Scheduler	0.55		

Every scheduler is capable of issuing one instruction per cycle, but for this kernel each scheduler only issues an instruction every 1.8 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 8 warps per scheduler, this kernel allocates an average of 7.58 active warps per scheduler, but only an average of 1.45 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to highly different execution durations per warp. Reducing stalls indicated on the [Warp State Statistics](#) and [Source Counters](#) sections can help, too.



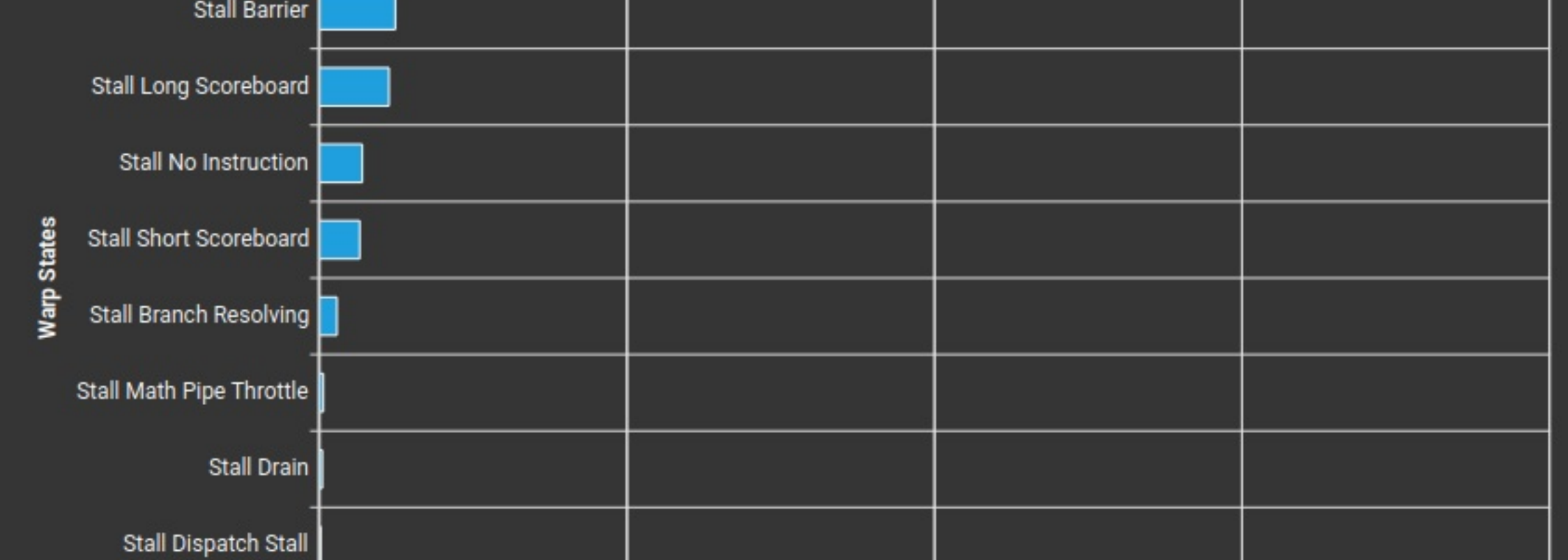
Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction type implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Warp Cycles Per Issued Instruction [cycle]	13.76	Avg. Active Threads Per Warp	31.58
Warp Cycles Per Executed Instruction [cycle]	1.45	Avg. Not Predicted Off Threads Per Warp	31.29

On average, each warp of this kernel spends 6.1 cycles being stalled waiting for an MIO instruction queue to be not full. This represents about 44.2% of the total average of 13.8 cycles between issuing two instructions. This stall reason is high in cases of utilization of the MIO pipelines, which include special math instructions, dynamic branches, as well as shared memory instructions. When caused by shared memory accesses, trying to use fewer but wider loads can reduce pipeline pressure.

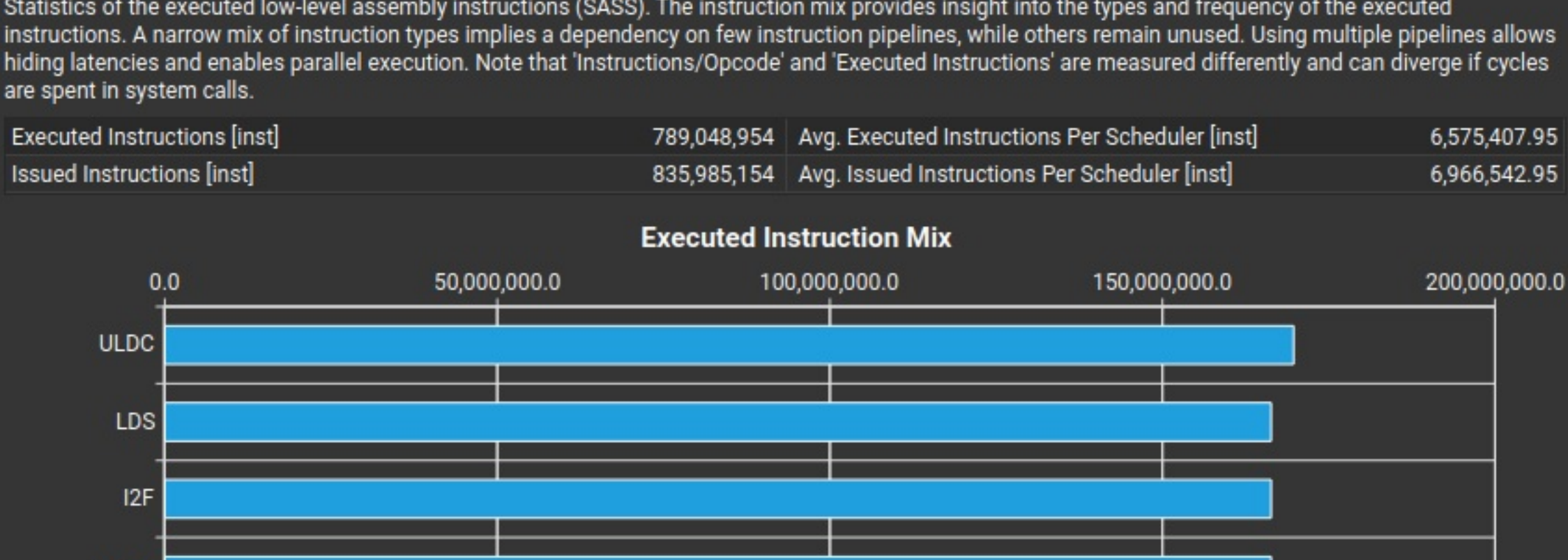
Check the [Source Counters](#) section for the top stall locations in your source based on sampling data. The [Kernel Profiling Guide](#) provides more details on each stall reason.



Instruction Statistics

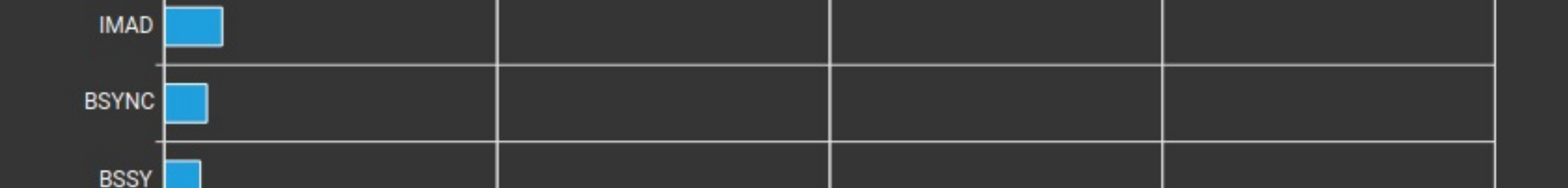
Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [Inst]	889,048,954	Avg. Executed Instructions Per Scheduler [Inst]	6,575,407.95
Issued Instructions [Inst]	83,955,154	Avg. Issued Instructions Per Scheduler [Inst]	6,966,542.95



NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.



The system does not have any NVLink connections.

Detailed Tables

Tables containing with properties for each NVLink.

Logical NVLink Properties

The system does not have any NVLink connections.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	57,600	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	31	Static Shared Memory Per Block [kbyte/block]	1.6
Block Size	256	Dynamic Shared Memory Per Block [byte/block]	0
Threads [SM]	14,745,600	Driver Shared Memory Per Block [byte/block]	0
Waves Per Thread	480	Shared Memory Configuration Size [Kbyte]	32.77

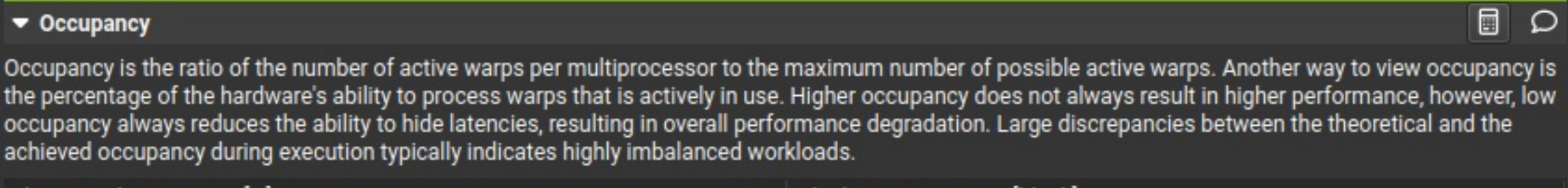
Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

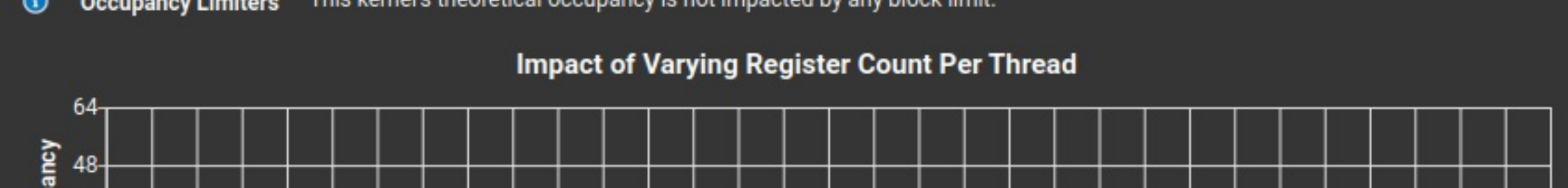
Theoretical Active Warps per SM [warp]	100	Block Limit Registers [block]	8
Theoretical Occupancy [%]	32	Block Limit Shared Mem [block]	25
Achieved Occupancy [%]	94.87	Block Limit Warps [block]	4
Achieved Active Warps Per SM [warp]	30.36	Block Limit SM [block]	16

This kernel's theoretical occupancy is not impacted by any block limit.

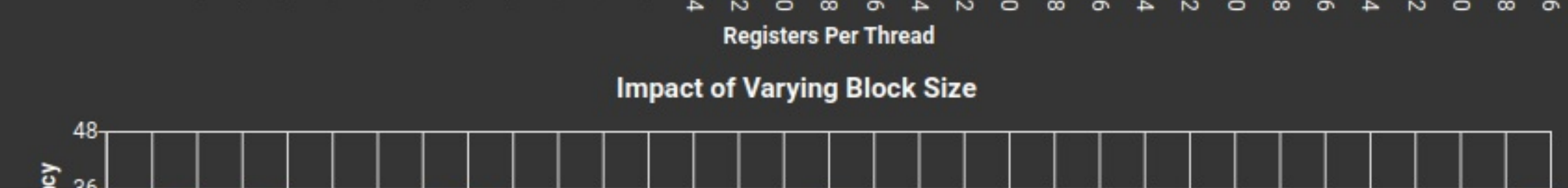
Impact of Varying Register Count Per Thread



Impact of Varying Block Size



Impact of Varying Shared Memory Usage Per Block



Source Counters

All

Source counters including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [Inst]	25,294,418	Branch Efficiency [%]	94.52
Branch Instructions Ratio [%]	0.03	Avg. Divergent Branches	8,226.82

This kernel has uncoalesced global accesses resulting in a total of 5326186 excessive sectors (61% of the total 8713854 sectors). Check the L2 Theoretical Sectors Global Excessive table for the primary source locations. The [CUDA Programming Guide](#) had additional information on reducing uncoalesced device memory accesses.

L2 Theoretical Sectors Global Excessive

Location	Value	Value (%)
gaussian.cu:106 (0x7bc30525a750 in gaussian_...)	1,946,218	37
gaussian.cu:106 (0x7bc30525a680 in gaussian_...)	1,946,218	37
gaussian.cu:106 (0x7bc30525a630 in gaussian_...)	972,950	18
gaussian.cu:127 (0x7bc30525b1d0 in gaussian_...)	460,800	9

Warp Stall Sampling (All Cycles)

Location	Value	Value (%)
gaussian.cu:123 (0x7bc30525a6b0 in gaussian_...)	16,552	0
gaussian.cu:106 (0x7bc30525a680 in gaussian_...)	12,471	0
gaussian.cu:117 (0x7bc30525a630 in gaussian_...)	12,401	0
gaussian.cu:123 (0x7bc30525a6d0 in gaussian_...)	10,270	0
gaussian.cu:123 (0x7bc30525a6c0 in gaussian_...)	9,185	0

Warp Stall Sampling (Not-Issued Cycles)

Location	Value	Value (%)
gaussian.cu:123 (0x7bc30525a6b0 in gaussian_...)	6,972	0
gaussian.cu:106 (0x7bc30525a680 in gaussian_...)	6,022	0
gaussian.cu:117 (0x7bc30525a630 in gaussian_...)	5,881	0
gaussian.cu:123 (0x7bc30525a6d0 in gaussian_...)	4,417	0
gaussian.cu:106 (0x7bc30525a6c0 in gaussian_...)	4,169	0

Most Instructions Executed

Location	Value	Value (%)
gaussian.cu:117 (0x7bc30525b180 in gaussian_...)	8,755,200	1
gaussian.cu:123 (0x7bc30525b170 in gaussian_...)	8,755,200	1
gaussian.cu:123 (0x7bc30525b160 in gaussian_...)	8,755,200	1
gaussian.cu:117 (0x7bc30525b150 in gaussian_...)	8,755,200	1
gaussian.cu:123 (0x7bc30525b140 in gaussian_...)	8,755,200	1